Active reconnaissance and Information Gathering

7CSEF006W

Dr Ayman El Hajjar

Academic Year: 2025/26

---

1 Lab environment

PLEASE READ

- You will find that the lab is showing activities on the Linux server **192.168.144.101**, however this is for one server.
- For your group assignment, you are expected to conduct all activities on both the Linux and the Windows servers.

`You will be provided with the Windows server details by the end of Week 5.`

- Below are the IP addresses I am using for my machines. Your machines can have different IP addresses:
  - Kali Linux IP address: **192.168.144.103**
  - Linux Server IP address: **192.168.144.101**

- In some activities you need **only** your Kali Linux machine to be connected to the Internet.

- Resolveconf is a DNS name server. On the new Kali Linux (2020) it doesn't come pre-installed. To install it:

```
sudo apt-get install resolvconf
```

- It is needed for `nmap` to identify the domain name resolver for your machine.

- Your DHCP server should have given you the correct DNS address.
  - Let us enable `resolvconf` service and then start it:

```
sudo systemctl enable resolvconf.service
```

```
sudo systemctl start resolvconf.service
```

```
sudo systemctl status resolvconf.service
```

Sometimes Linux fails to obtain an IP address from the DHCP server. A simple invoke of `dhclient` allows it to dynamically obtain one:

```
sudo dhclient
```

↳ This typically happens when you change the network interface from NAT to host-only.

↳ You can also manually disconnect and reconnect the network interface.

1.1 Taking advantage of robots.txt

One step further into reconnaissance is to figure out if there are any pages or directories on the site that are hidden from regular users. For example, a login page to the intranet or the **content management systems (CMS)** administration.

Finding such pages expands our testing surface considerably and can provide important clues about the application and its infrastructure.

We will use the **robots.txt** file to discover some files and directories that may not be linked anywhere in the main application.

Open the browser and go to: `http://192.168.144.101/drupal/`

Now append `robots.txt` to the URL: `http://192.168.144.101/drupal/robots.txt`

This file tells search engines which directories should not be indexed. For example, the `modules` folder is typically hidden from crawlers.

Try browsing to: `http://192.168.144.101/drupal/modules/`

How this tool works

The `robots.txt` file is used by web servers to tell search engines which directories or files they should or should not index.

From an attacker's perspective, this file reveals directories and files that may be hidden from regular users using "security through obscurity."

View the file directly here: `http://192.168.144.101/drupal/robots.txt`

You should spend time exploring both servers and identifying what services and directories are accessible.

Reconnaissance and information gathering form an essential stage in every penetration test or security assessment.

The more information you have about your target, the more options you'll have when searching for vulnerabilities.

1.2 Finding Files and Folders Using DirBuster

DirBuster is a tool used to discover, by brute force, the existing files and directories on a web server. We will use it to search for specific paths based on a custom dictionary.

Prepare a dictionary file with common file and directory names to test.

DirBuster Overview

DirBuster acts as both a crawler and a brute-forcer. It follows links on discovered pages but also attempts paths from the provided dictionary.

To determine if a file exists, DirBuster checks the server's response codes:

- `200. OK: File exists and is readable.`
- `404. File not found: The file does not exist on the server.`
- `301. Moved permanently: URL has been redirected.`
- `401. Unauthorized: Requires authentication.`

403. Forbidden: Request was valid but the server refused access.

Create a file named `dictionary.txt` containing the following entries:

- info
- server-status
- server-info
- cgi-bin
- robots.txt
- phpmyadmin
- admin
- login
- chat

To launch DirBuster:

- Navigate to: **Kali Linux → Web Application Analysis → Web Crawlers & Directory Bruteforce → DirBuster**.
- Set the target URL to: `http://192.168.144.101:80`
- Set the number of threads to **20**.
- Select **List-based brute force** and click **Browse** to choose `dictionary.txt`.
- Uncheck the **Be Recursive** option.

↳ Click **Start**.

In the **Results** tab, DirBuster should identify files like `server-status` and `phpmyadmin`. A `200 OK` response confirms the resource exists and is accessible.

Repeat this process for all web applications on both servers, as these findings will be valuable later.

1.3 DNS Enumeration

Enumeration is the process of gathering information from a network. In this section, we examine **DNS enumeration** — the process of locating DNS servers and DNS entries for an organisation.

DNS enumeration provides critical data such as hostnames, usernames, computer names, IP addresses, and more.

Do not perform DNS enumeration against external websites without proper authorisation.

The **dnsenum** tool is pre-installed in Kali Linux and can be used for DNS enumeration.

Commands for DNS Enumeration

```
dnsenum --enum www.hackthissite.org
dnsenum --enum cwscenario.uk
```

These commands output details such as:

↳ Hostnames

↳ Name servers

↳ Mail servers

↳ Zone transfers (if successful)

Explore additional features of `dnsenum` using the `man` command:

```
man dnsenum
```

Note

The tools we used here are not comprehensive. Kali Linux provides many other DNS-related utilities that you are encouraged to explore.

2 The Network Mapper (Nmap)

One of the most important stages of reconnaissance is identifying open services, running software, and host configurations.

Nmap (Network Mapper) is one of the most widely used tools for this purpose.

It can detect:

- Live hosts
- Open TCP and UDP ports
- Firewalls and filtering mechanisms
- Running services and software versions
- Potential vulnerabilities through scripts

2.1 Default Scanning with Nmap

Let's begin by running a simple Nmap scan against an external host. Make sure your Kali Linux machine is connected to the Internet.

Default Scan - Internet

```
nmap scanme.nmap.org
```

This command performs a basic TCP **connect scan** without requiring root privileges.

The results are limited compared to privileged scans, but you can still identify open ports and services.

Now, let us perform a local scan on our vulnerable server. Ensure your Kali machine is switched to the **Host-Only** network mode.

Default Scan - Local

```
nmap 192.168.144.101
```

Review the scan results to identify open services and their associated ports.

2.2 Port Scans

By default, Nmap performs a simple port scan on the target host, identifying open ports, services, and their states.

Nmap categorises ports into the following states:

↳ **Open**: A service is listening on this port.

↳ **Closed**: The port is accessible but no service is listening.

↳ **Filtered**: Nmap cannot determine the state due to packet filtering (e.g. firewall rules).

↳ **Unfiltered**: The port is accessible, but the state could not be determined.

↳ **Open/Filtered**: The port may be open or filtered — Nmap cannot distinguish between the two.

↳ **Closed/Filtered**: The port may be closed or filtered — Nmap cannot determine the state.

You can also specify custom DNS servers while scanning to avoid using the system default:

```
nmap -dns-servers 8.8.8.8,8.8.4.4 scanme.nmap.org
```

2.2.1 Scanning Specific Port Ranges

By default, Nmap scans the top 1000 TCP ports. To scan other ports, you must specify them explicitly.

Use the **-p** flag followed by a range or a single port number:

```
nmap -p 80-3000 scanme.nmap.org
```

```
nmap -p 80 nmap.org
```

You can scan any custom range, including all 65,535 possible ports if needed.

2.2.2 Scanning IP Ranges

Nmap supports scanning multiple hosts by specifying IP ranges in several formats:

```
nmap 192.168.144.100-103
```

This scans all hosts from `192.168.144.100` to `192.168.144.103`.

↳ You can also use wildcards to scan an entire subnet:

```
nmap 192.168.144.*
```

↳ Alternatively, you can use CIDR notation:

```
nmap 192.168.144.0/24
```

2.3 Russian Roulette - Random Target Scanning

Nmap allows you to scan a random list of hosts using the **-iR** flag.

```
sudo nmap -iR 100
```

This example scans 100 random hosts on the Internet.

↳ You can combine random scans with other flags. For example, scanning unlimited IPs for open NFS shares:

```
nmap -p2049 --open -iR 0
```

 Use this option cautiously — scanning unknown networks may violate acceptable use policies or local laws.

2.4 Logging Scans

When performing extensive reconnaissance, it is good practice to log results for later analysis.

Nmap supports multiple output formats, including `.xml`, `.nmap`, and `.gnmap`.

↳ Use the **-oA** flag to save results in all three formats simultaneously:

```
nmap scanme.nmap.org -oA sitelog
```

```
nmap 192.168.144.101 -oA servers-log
```

This will produce:

↳ `sitelog.nmap` – Readable scan results

↳ `sitelog.xml` – XML structured output for automation

↳ `sitelog.gnmap` – Greppable results for parsing

2.5 Tracing routes

Ping scans can include trace route information to the targets. Use the Nmap option **--traceroute** to trace the route from the scanning machine to the target host:

```
sudo nmap -sn --traceroute cwscenario.site
```

2.6 Host detection methods

A simple way would be by performing a **ping** on the target network. Of course, this can be rejected or known by a host, and we don't want that.

In order to scan a host effectively it is essential to identify whether they are "alive" or "online".

Many administrators hide their systems from the internet and they can appear offline until further probed.

One way is using a ping (ICMP echo request).

You can run a ping-only sweep using Nmap with **-sn**. This forces Nmap to simply check whether the host is alive regardless of ports.

Live hosts in a network

```
nmap -sn -n 192.168.144.*
```

We scanned the whole range `192.168.144.0` to `192.168.144.255` for live hosts. The `*` represents the whole class C range.

2.7 Agnostic scan

Even with the previous scan, a system can still hide from ping sweeps.

Nmap offers an agnostic method using **-Pn**.

↳ In the previous command, Nmap was using default port addresses, which might skip services running above port 1000.

```
nmap -Pn scanme.nmap.org
```

↳ Compare your results with a simple ping scan for the same site:

```
nmap -sn scanme.nmap.org
```

↳ Try the testing site:

```
nmap -Pn cwscenario.site
```

List scan

You can add a list scan flag **-sL** to get reverse DNS lookups and understand live hosts. For example:

```
nmap -sL cwscenario.site
```

2.8 Scanning UDP services

Since UDP is connectionless, scanning is harder (often slower and less reliable than TCP scans).

Many important services run on UDP; it is essential to scan them too.

To scan with UDP, invoke **-sU**; **It will be slow. Make sure you do this at home when you have plenty of time.** UDP scans also require elevated privilege.

UDP services

```
sudo nmap -sU cwscenario.site
sudo nmap -sU 192.168.144.102
```

2.9 Scanning TCP services

Two basic TCP scans in Nmap are the connect scan **-sT** and the SYN (stealth) scan **-sS**. These are also called full and half connection scans.

```
-sT
```

flag - TCP connect scan is the default TCP scan type when SYN scan is not an option (no raw packet privileges or IPv6). It uses the system's connect call and may be noisy (full connections).

-sS flag - TCP SYN scan. Fast, relatively unobtrusive (never completes connections), and distinguishes open/closed/filtered states clearly. Often called half-open scanning.

TCP scans services

```
sudo nmap -sT cwscenario.site
sudo nmap -sT 192.168.144.102
sudo nmap -sS cwscenario.site
sudo nmap -sS 192.168.144.102
```

You can also specifically use TCP SYNC flag by invoking **-PS** flag.

For example:

```
nmap -sn -PA 192.168.144.1/24
```

↳ We use **-sn** to stop port scanning and only identify hosts.

↳ The **-PA** flag tells Nmap to use an ACK ping scan (expects acknowledgements, so only online hosts respond). Nmap sends an empty TCP packet and waits for an ACK for each host.

You can also specifically use TCP ACK flag by invoking **-PS** flag.

For example:

```
nmap -sn -PS 192.168.144.1/24
```

↳ We use **-sn** to stop port scanning and only identify hosts.

↳ The **-PS** flag tells Nmap to use a TCP SYN ping scan (send SYN to port 80, receive RST if closed, SYN/ACK if open, then RST to reset).

- You can specify ports with **-PS**:

```
nmap -sn -PS80,21 192.168.144.1/24
```

2.9.1 Special TCP services

- If an IDS drops SYN/full scans, consider alternative TCP probes: **FIN (-sF)**, **Xmas Tree (-sX)**, and **NULL (-sN)**.

- FIN scan:

```
sudo nmap -sF cwscenario.site
```

- Xmas Tree scan uses FIN, URG, and PSH flags: **-sX**.

- Null scan sets no flags: **-sN**.

These special scans generally do not work against Microsoft Windows hosts.

2.10 Scanning using ICMP packets

- Ping scans determine if a host is online. ICMP echo request (**-PE**) is designed for this.

ICMP echo request

```
sudo nmap -PE cwscenario.site
sudo nmap -PE 192.168.144.102
```

2.11 discovering hosts with ARP ping scans

- ARP ping scans are the most effective way of detecting hosts on LANs and are preferred on local Ethernet.

- Nmap uses **-PR** for ARP discovery and may choose it automatically on LANs.

Hosts discovery using ARP scans

```
sudo nmap -sn -PR 192.168.144.101
```

- Note the network interface MAC address returned for your target IP address.
- You can see the full ARP resolution exchange using the **--packet-trace** flag:

```
sudo nmap -sn -PR --packet-trace 192.168.144.101
```

Why this is useful

- If the server employs an IDS, now that we know the target MAC we could spoof it to evade detection.

```
sudo nmap -sn -PR -spoof-mac 08:00:27:88:93:F6 192.168.144.101
```

- Note that the network interface MAC address was returned for your target IP address.
  - ↳ You can even see the whole ARP resolution request by using the **--packet-trace** flag:

```
sudo nmap -sn -PR --packet-trace 192.168.144.101
```

Note this is my virtual machine MAC address; yours will be different.

2.12 Fingerprinting

2.12.1 Fingerprinting for Operating system detection

- Determining the services running on specific ports will ensure a successful pentest on the target network. It will also remove any doubts left resulting from the OS fingerprinting process.

- Nmap **-O** flag works by comparing the fingerprint identified on a host by a large database of fingerprints for services, servers and operating systems used worldwide.

- It uses the Common Platform Enumeration (CPE) as the naming scheme for service and operating system detection. This convention is used in the information security industry to identify packages, platforms, and systems.

- The complete documentation of the tests and probes sent during OS detection can be found at OS detection methods site

Operating system detection

```
nmap -O 192.168.144.102
```

2.12.2 Fingerprinting for service detection

💡Running a service version scan is very simple; all we need to do is add an additional flag, **-sV**. This means that we're conducting a service version scan, which can demonstrate which version of each software is running.
This is particularly useful if someone is running a service on a non-default port (that does not match up with /etc/services)—in such instances, it's even more important to be able to figure out exactly what's running.

Default scan - Internet

`service version scans`
For example to scan the nmap site:

```
nmap -sV -n scanme.nmap.org
```
and to scan the metasploitable server:

```
nmap -sV -n 192.168.144.101
```
Now let us compare the results of this command to the previous one.
Can you imagine what is the importance of the information you obtained from this command in comparison with the previous one?

Understanding the reason flag

- If you think of the TCP three-way handshake, you can ask Nmap to give you the reason behind the different status of the flags you can have **open, closed, and filtered.**
- Read these definitions on port flags on the Nmap site
- To scan metaspoloitable:

```
nmap -sV -n --reason 192.168.144.101
```

2.13 Optimising scans

2.13.1 Increasing version detection intensity

- You can increase or decrease the amount of probes to use during version detection by changing the intensity level of the scan with the argument --version-intensity [0-9], as follows:

```
nmap -sV --version-intensity 9 192.168.144.101
```

- This Nmap option is incredibly effective against services running on nondefault ports due to configuration changes.

2.13.2 Aggressive detection mode

Nmap has a special flag to activate aggressive detection, namely **-A**. Aggressive mode enables OS detection (**-O**), version detection (**-sV**), script scanning (**-sC**), and traceroute (−−**traceroute**).

This mode sends a lot more probes, and it is more likely to be detected, but it provides a lot of valuable host information.

You can try aggressive detection with the following command:

```
nmap -A 192.168.144.101
```

2.13.3 Configuring OS detection

If OS detection fails, you can use −−osscan-guess to force Nmap to guess the Operating system.

```
nmap −−osscan-guess 192.168.144.101
```

This Nmap option is incredibly effective against services running on nondefault ports due to configuration changes.

2.13.4 Increasing verbosity in scans

Verbosity lets timing, parallelism, and internal debugging information display straight to the console while scans run.

This can be useful to figure out when we need to try to optimize scans in one of the several ways.

When running a scan in increased verbosity, you can also hit Enter to see how far the scan has progressed, and how far it has to go before completing its current target file.

There are several different levels of verbosity:

- The first level of verbosity gives basic information about a scan's progress and can be invoked by using the **-v** flag.
- The second level of verbosity gives more details, including network and packet information, and can be invoked by using **-vv** as a flag.
- Lastly, triple verbosity—which gives the most information—can be invoked with the **-v3** or **-vvv** flag.
- If you want to make Nmap less verbose than normal, you can also use the −−reduce-verbosity flag.

2.13.5 Nmap timing optimisation

The easiest way to make a scan run faster is to use the built-in timing flags. These flags are invoked using **-T** and a number from 1 (slowest) to 5 (fastest).

The default scanning speed is **-T3**, right in the middle.

There are risks with significantly faster scanning, since it may create unreliable results.

If intrusion detection systems or prevention systems are running, slower scans may help evade detection.

```
nmap -T2 192.168.144.101
```

2.13.6 Increasing and decreasing parallelism

You can increase or decrease the number of scans running in parallel to make your scans more efficient.

For example, by increasing the value of −−**min-parallelism** up to 10 or 12, you can force Nmap to scan at least that fast.

On the other hand, you can lower the value of −−**max-parallelism** as low as 1 to make scans more controlled.

Parallelism optimization

```
nmap -sU −−min-parallelism 10 192.168.144.101
```

2.13.7 Dealing with stuck hosts

Sometimes hosts can take a very long time to respond. If you are dealing with a large number of hosts, it is better to skip it and come back later.

You can use the −−**host-timeout** flag and specify the value in minutes before the scan moves to another host.

Stuck hosts

```
nmap -Pn 10 192.168.144.* −−host-timeout 1m
```

2.14 List of targets

Many times we want to scan a list of IP addresses and not an entire subnet.

We can use any text editor and create a list of IP addresses and "feed" it to Nmap.

This is the list of IP addresses we want to scan.

Service fingerprinting

`gedit scanlist.txt`

↳ Inside this file, put several IP addresses you want to scan.

↳ Type **nmap -iL scanlist.txt** to scan all IP addresses in the list.

You need to be in the same location as your `scanlist.txt` file for the command to run, or provide the absolute path.

Nmap is a port scanner, which means it sends packets to a number of TCP or UDP ports on the indicated IP address and checks for responses.

If there is a response, the port is open, meaning a service is running on that port.

In the first command, with the `-sn` parameter, we instructed Nmap to only check if the server was responding to ICMP requests (pings).

Our server responded, so it is alive.

The second command is the simplest way to call Nmap: it only specifies the target IP address.

This pings the server, and if it responds, Nmap sends probes to 1,000 TCP ports and reports the ones that responded.

The third command extends this by:

↳ **-sV** asks for the banner (header or self-identification) of each open port found, used for version detection.

↳ **-O** tells Nmap to try to guess the operating system running on the target using the open ports and version information.

For each command we used, you can refer to the **man** command to explore additional options.

Other useful Nmap parameters

↳ **-sT**: Forces a full TCP connect scan instead of the default SYN scan. Slower but useful for avoiding detection by IDS.

↳ **-Pn**: Skips the host discovery phase and assumes the host is up. Useful if ICMP requests are blocked.

↳ **-v**: Enables verbose mode. Use multiple times (`-vv` or `-vvv`) for more detail.

↳ **-p N1,N2,...,Nn**: Scans specific ports or ranges, e.g., `-p 21,80-90,137`.

↳ **--script=script_name**: Runs an Nmap script from the NSE repository.

↳ For available scripts, visit: https://nmap.org/nsedoc/scripts

2.15 Using Nmap Scripting Engine (NSE) for Reconnaissance

- The Nmap scripting engine (NSE) is one of Nmap's most powerful and flexible features.
- It allows users to write their own scripts and share them for networking, reconnaissance, vulnerability detection, and exploitation.
- These scripts can be used for:
  - Network discovery
  - More sophisticated and accurate OS version detection
  - Vulnerability detection
  - Backdoor detection
  - Vulnerability exploitation
- The NSE framework runs code written in the Lua programming language.
- Lua is lightweight, fast, and widely used for automation and game scripting.
- For now, we will be using only pre-written scripts.
- The official Nmap script repository: Nmap NSE Documentation
- Scripts are grouped into categories based on their purpose:

Nmap NSE script categories

- **Auth**: Authenticate to services and verify credentials.
- **Broadcast**: Broadcasts to detect active services on the network.
- **Brute**: Performs brute-force or dictionary-based attacks.
- **Default**: Default scripts executed during a scan.
- **Discovery**: Enumerates sensitive information from hosts and services.
- **DoS**: Tests for denial-of-service vulnerabilities (use cautiously).
- **Exploit**: Executes known exploits on vulnerable targets.
- **External**: Queries third-party services like DNS blacklists.
- **Fuzzer**: Sends random data to services to find flaws.
- **Intrusive**: May cause disruptions or impact services.
- **Malware**: Scans for malware or malicious services.
- **Safe**: Verified safe scripts that do not disrupt services.
- **Version**: Detects specific software versions and vulnerabilities.
- **Vuln**: Identifies known vulnerabilities in services.

2.15.1 Running Nmap Scripts

- Before running NSE scripts, update the local scripts database:

```
sudo nmap --script-updatedb
```

- Once updated, scripts can be run by group or individually.
- For example, to run the **default** scripts:

```
nmap 192.168.144.101 --script default
```

- To scan all scripts related to web servers (http):

```
nmap 192.168.144.101 --script "http-*"
```

- To find a specific script, navigate to the scripts folder:

```
cd /usr/share/nmap/scripts
```

  ↳ List all scripts:

```
ls
```

  ↳ Search for a specific keyword within script names:

```
ls | grep drupal
```

- Nmap includes scripts for testing Web Application Firewalls (WAF). For example:

```
nmap -p 80,443 --script=http-waf-detect 192.168.144.101
```

- You can also run WAF fingerprinting against external websites:

```
nmap -p 80,443 --script=http-waf-fingerprint cwscenario.site
```

- You can combine NSE scripts with additional Nmap flags for powerful scanning.
- For example, running a DNS brute-force script at maximum speed:

```
sudo nmap --script dns-brute cwscenario.site -T5
```

Use aggressive timing carefully. High speeds may trigger IDS or be blocked.

3 Other scanners tools

- Although Nmap is the most popular, it is not the only port scanner available. Depending on your objectives, other tools may be faster or provide more flexibility.
- Kali Linux includes several alternatives, such as:

- ↳ unicornscan
- ↳ hping3
- ↳ masscan
- ↳ Metasploit scanning modules

Using **masscan**, we can also identify the application running on a specific port or a range of ports:

```
sudo masscan 192.168.144.101 -p80
```

Example of scanning all open ports using Unicornscan:

```
sudo unicornscan -Iv 192.168.144.101:a
```

Example of using hping3 for a simple SYN scan:

```
sudo hping3 -S -p 80 -c 3 192.168.144.101
```

Unicornscan is a fast asynchronous TCP/UDP scanning tool that can quickly discover open ports.

hping3 is a packet crafting tool. In this example, we are using it to send three SYN packets to port 80 of the target host to verify if it responds.

These tools are complementary to Nmap and can be useful for bypassing firewalls, cross-validating results, and performing advanced scans.